

Name: AAVISHKAR SINGH AAVISHKAR SINGH | Reg. No: RA2511003011024

Unit 3 : Session 5 : SLO 1 — Booth's Multiplication Algorithm (Setup)

■ 1. Fill in the Blanks (Remembering)

- The single bit register Q-1 is initially set to 0.
- The accumulator (A) and Q-1 are initialized to 0.
- The sequence counter (SC) is set to the number of BITS in the multiplier.
- The algorithm examines the last two bits of the Q and Q-1 registers to determine the next action.

■ 2. True or False (Understanding)

- a. If $Q_0 Q_{-1} = '10'$, perform $A = A - M \rightarrow \text{TRUE}$
- b. An arithmetic right shift is performed on A, Q and Q-1 in every cycle $\rightarrow \text{TRUE}$
- c. The algorithm stops when the sequence counter becomes 1 $\rightarrow \text{FALSE}$ (it stops when SC becomes 0)
- d. If $Q_0 Q_{-1} = '01'$, perform $A = A + M \rightarrow \text{TRUE}$
- e. Booth's algorithm is only effective for multiplying positive numbers $\rightarrow \text{FALSE}$

■ 3. Matching (Applying)

- a. 0 0 $\rightarrow$ 3. No operation
- b. 0 1 $\rightarrow$ 2. $A = A + M$
- c. 1 0 $\rightarrow$ 1. $A = A - M$
- d. 1 1 $\rightarrow$ 3. No operation

■ 4. Step-by-Step Analysis (Analyzing): $(+7) \times (-3)$, $M = 0111$, $Q = 1101$, $A = 0000$, $Q_{-1} = 0$

- Step 1a: $Q_0 Q_{-1} = '10' \rightarrow$ perform $A = A - M$: $A = 0000 - 0111 = 1001$.
- Step 1b: after arithmetic right shift $\rightarrow A = 1100$, $Q = 1110$, $Q_{-1} = 1$.
- Step 2c: new $Q_0 Q_{-1} = '01' \rightarrow$ perform $A = A + M$: $A = 1100 + 0111 = 0011$.
- Step 2d: after the second arithmetic right shift $\rightarrow A = 0001$, $Q = 1111$, $Q_{-1} = 0$.

■ 5. Algorithm Explanation (Creating)

- Booth's algorithm multiplies signed 2's-complement numbers efficiently by handling blocks of 1s in the multiplier. Instead of adding for every 1, it performs one subtraction at the start of a run of 1s (pattern 10) and one addition at the end (pattern 01).
- This reduces the number of arithmetic operations for long strings of 1s, and because it works directly on 2's-complement values it handles negative numbers with no extra sign logic, making multiplication faster and simpler in hardware.